

Advanced Java Programming

Assignment No. 4 – Unit 4: Hibernate & Spring Framework

■■ Hibernate Framework – Introduction & Architecture ■■

Hibernate is an ORM (Object-Relational Mapping) framework for Java. It maps Java classes to database tables and Java data types to SQL data types, eliminating the need for manual JDBC code.

Component	Description
Configuration	Reads hibernate.cfg.xml or annotations; builds SessionFactory
SessionFactory	Heavy object; created once per application; thread-safe; creates Sessions
Session	Lightweight, short-lived; represents a single unit of work with DB
Transaction	Wraps DB operations; ensures ACID properties
Query / Criteria	HQL (Hibernate Query Language) or Criteria API for DB queries
Entity Class	POJO mapped to DB table using @Entity, @Table, @Column annotations

Hibernate Annotations Quick Reference:

@Entity - Marks class as Hibernate entity (maps to a table)
@Table(name="...") - Specifies table name (optional if same as class name)
@Id - Marks field as primary key
@GeneratedValue - Auto-generates primary key value
@Column(name="...") - Maps field to a specific column
@Transient - Field is NOT mapped to any column
@OneToMany, @ManyToOne, @ManyToMany - Relationship mappings

Required JAR files / Maven Dependencies:

```
<!-- pom.xml Maven Dependencies -->
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>5.6.15.Final</version>
</dependency>
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.30</version>
</dependency>

<!-- OR add these JARs to classpath manually:
hibernate-core-5.x.jar, hibernate-commons-annotations.jar,
antlr-2.7.7.jar, dom4j-2.1.3.jar, javassist-3.x.jar,
mysql-connector-java-8.x.jar, jboss-logging.jar -->
```

Common File 1 — hibernate.cfg.xml (used in all Q1–Q4):

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
  "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
  "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
  <session-factory>

    <!-- Database Connection Settings -->
    <property name="hibernate.connection.driver_class">
      com.mysql.cj.jdbc.Driver
```


[illegible]

```
// File: HibernateUtil.java
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateUtil {

    // Single SessionFactory for the entire application
    private static final SessionFactory sessionFactory = buildSessionFactory();

    private static SessionFactory buildSessionFactory() {
        try {
            // Create SessionFactory from hibernate.cfg.xml
            return new Configuration()
                .configure("hibernate.cfg.xml") // reads hibernate.cfg.xml
                .buildSessionFactory();
        } catch (Throwable ex) {
            System.err.println("SessionFactory creation failed: " + ex);
            throw new ExceptionInInitializerError(ex);
        }
    }

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }

    public static void shutdown() {
        // Close caches and connection pools
        getSessionFactory().close();
    }

}
```

Q1. Hibernate program to connect to a database and retrieve all records from a table using annotations.

Concept:

Use **session.createQuery()** with **HQL (Hibernate Query Language)** to retrieve records. HQL works on entity class names (not table names). Use **list()** to get all records.

RetrieveAllStudents.java:

```
// File: RetrieveAllStudents.java
import org.hibernate.*;
import org.hibernate.cfg.Configuration;
import java.util.List;

public class RetrieveAllStudents {
    public static void main(String[] args) {

        // Step 1: Create Configuration and build SessionFactory
        SessionFactory factory = new Configuration()
                                .configure("hibernate.cfg.xml")
                                .addAnnotatedClass(Student.class)
                                .buildSessionFactory();

        // Step 2: Open a Session
        Session session = factory.getCurrentSession();

        try {
            // Step 3: Begin transaction
            session.beginTransaction();

            // Step 4: HQL Query to retrieve all students
            // Note: "Student" is the Entity class name, NOT the table name
            List<Student> students = session
                .createQuery("FROM Student ORDER BY id", Student.class)
                .getResultList();

            // Step 5: Display results
            System.out.println("
===== All Students =====");
            System.out.printf("%-4s %-20s %-4s %-10s %-25s%n",
                "ID", "Name", "Age", "Course", "Email");
            System.out.println("-".repeat(65));

            for (Student s : students) {
                System.out.printf("%-4d %-20s %-4d %-10s %-25s%n",
                    s.getId(), s.getName(), s.getAge(),
                    s.getCourse(), s.getEmail());
            }
            System.out.println("-".repeat(65));
            System.out.println("Total Records: " + students.size());

            // Step 6: Commit transaction
            session.getTransaction().commit();

        } finally {
            factory.close();
        }
    }
}
```

HQL vs SQL Comparison:

SQL (uses table/column names):

```
HQL (uses Entity class/field names):
FROM Student ORDER BY id
FROM Student WHERE course = 'MCA'
FROM Student WHERE age > 20 ORDER BY name
```

```
Other HQL examples:
    session.createQuery("FROM Student WHERE course=:c", Student.class)
        .setParameter("c", "MCA")
        .getResultList();

session.get(Student.class, 1);           // Get by primary key
session.load(Student.class, 1);         // Lazy load by primary key
```

```
Hibernate:
    select student0_.id as id, student0_.name as name,
    student0_.age as age, student0_.course as course
    from students student0_ order by student0_.id
```

ID	Name	Age	Course	Email
1	Rahul Sharma	20	BCA	rahul@example.com
2	Priya Patil	21	MCA	priya@example.com
3	Amit Joshi	22	BSc CS	amit@example.com

Note: Hibernate automatically generates and executes the SQL. `hibernate.show_sql=true` prints the generated SQL in console — very useful for debugging.

```
// File: InsertStudent.java
import org.hibernate.*;
import org.hibernate.cfg.Configuration;

public class InsertStudent {

    public static void main(String[] args) {

        // Build SessionFactory
        SessionFactory factory = new Configuration()
                                .configure("hibernate.cfg.xml")
                                .addAnnotatedClass(Student.class)
                                .buildSessionFactory();

        Session session = factory.getCurrentSession();

        try {

            // ■■ Insert Single Record ████████████████████████████████████████████
            session.beginTransaction();

            // Create new Student entity object
            Student student1 = new Student("Neha Kulkarni", 22, "MCA",
                                           "neha@example.com");

            // session.save() inserts the record and returns generated ID
```

```

    } catch (Exception e) {
        if (session.getTransaction() != null)
            session.getTransaction().rollback();
        System.out.println("Error: " + e.getMessage());
        e.printStackTrace();
    } finally {
        factory.close();
    }
}
}

```

save() vs persist() vs saveOrUpdate():

<code>session.save(obj)</code>	- Inserts new record; returns generated Serializable ID
<code>session.persist(obj)</code>	- Similar to <code>save()</code> ; does NOT return ID; JPA standard
<code>session.saveOrUpdate(obj)</code>	- Inserts if new (id=0), Updates if exists (id!=0)
<code>session.merge(obj)</code>	- JPA equivalent of <code>saveOrUpdate()</code>

```
// Hibernate State Diagram:
// Transient -> (save/persist) -> Persistent -> (commit/flush) -> DB INSERT
// Detached -> (merge/update) -> Persistent -> (commit/flush) -> DB UPDATE
```

Console Output:

```

Hibernate:
    insert into students (name, age, course, email) values (?, ?, ?, ?)

Record 1 inserted! Generated ID: 4
Saved: Student [id=4, name=Neha Kulkarni, age=22, course=MCA]

Hibernate: insert into students (name, age, course, email) values (?, ?, ?, ?)
Inserted: Raj Deshmukh    with ID: 5
Inserted: Sneha More      with ID: 6
Inserted: Kiran Patil     with ID: 7

All records inserted successfully!

```

Q3. Hibernate program to update a record in a database table using Hibernate session.

Concept:

To update: first load the entity using **session.get()**, modify its fields, then call **session.update()**. Alternatively, use HQL UPDATE query for bulk updates. Hibernate generates the UPDATE SQL automatically.

UpdateStudent.java:

[illegible]

[illegible]

} }

Hibernate: delete from students where course=?

HQL Bulk Delete: 1 row(s) deleted.

===== Remaining Records =====

Student [id=1, name=Rahul Sharma, age=20, course=BCA]

Student [id=2, name=Priya Patil, age=25, course=MSc IT]

Student [id=3, name=Amit Joshi, age=22, course=BSc CS]

Student [id=4, name=Neha Kulkarni, age=22, course=MCA]

Total remaining: 4

Complete Hibernate CRUD Summary:

Operation	Method	HQL Alternative
INSERT	session.save(obj) / persist(obj)	Not applicable (use save/persist)
SELECT	session.get(Class, id)	FROM Student WHERE ...
SELECT ALL	session.createQuery('FROM Student')	FROM Student ORDER BY name
UPDATE	session.update(obj)	UPDATE Student SET col=? WHERE ...
DELETE	session.delete(obj)	DELETE FROM Student WHERE ...

■ Spring Framework – Introduction & Dependency Injection ■

Spring Framework is a lightweight, comprehensive Java framework for enterprise application development. Its core feature is **IoC (Inversion of Control)** — the framework controls object creation and dependency injection instead of the programmer.

Spring Core Concepts:

IoC (Inversion of Control):

Traditional: Object A creates Object B manually (tight coupling)

Spring IoC: Spring container creates and injects Object B into A (loose coupling)

DI (Dependency Injection) – 3 types:

1. **Constructor Injection** : Dependencies passed through constructor parameters
2. **Setter Injection** : Dependencies set through setter methods (setXxx())
3. **Field Injection** : Dependencies injected directly into fields using @Autowired

Spring Container:

```
BeanFactory      : Basic IoC container (lightweight)
```

```
ApplicationContext : Advanced container (extends BeanFactory)
```

- `ClassPathXmlApplicationContext` : Loads from classpath XML
- `FileSystemXmlApplicationContext` : Loads from file system XML
- `AnnotationConfigApplicationContext` : Java-based config (@Configuration)

Required Maven Dependencies for Spring:

```
<!-- pom.xml -->
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>5.3.23</version>
</dependency>
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-core</artifactId>
  <version>5.3.23</version>
</dependency>
<!-- JARs: spring-core, spring-context, spring-beans,
      spring-aop, commons-logging -->
```

Q5. Spring program demonstrating constructor-based Dependency Injection.

Concept:

In **Constructor Injection**, the Spring container passes the dependency objects as constructor arguments when creating the bean. Dependencies are declared in the constructor and configured in XML with **<constructor-arg>** tags.

Step 1 — Engine.java (Dependency class):

```
// File: Engine.java
public class Engine {
    private String type;
    private int horsepower;

    // Constructor used by Spring for injection
    public Engine(String type, int horsepower) {
        this.type = type;
        this.horsepower = horsepower;
        System.out.println("Engine constructor called: " + type);
    }

    public void start() {
        System.out.println("Engine started: " + type +
            " (" + horsepower + " HP)");
    }
}
```

```
public String getType()    { return type; }
public int getHorsepower() { return horsepower; }
}
```

Step 2 — Car.java (Main bean with injected dependency):

[illegible]

Step 3 — applicationContext.xml (Spring Configuration):

```
<!-- File: applicationContext.xml (in src/ or resources/) -->
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- Bean 1: Engine (dependency) -->
    <!-- constructor-arg injects values into Engine(String, int) constructor -->
    <bean id="myEngine" class="Engine">
        <constructor-arg value="V8 Petrol" />
        <constructor-arg value="400" />
    </bean>

    <!-- Bean 2: Car (main bean) -->
    <!-- Spring injects: "Toyota", "Fortuner", and myEngine bean -->
    <bean id="myCar" class="Car">
        <constructor-arg value="Toyota" /> <!-- String brand -->
        <constructor-arg value="Fortuner" /> <!-- String model -->
        <constructor-arg ref="myEngine" /> <!-- Engine object -->
    </bean>

    <!-- Another car with different engine -->
    <bean id="sportsCar" class="Car">
        <constructor-arg value="BMW" />
        <constructor-arg value="M Series"/>
        <constructor-arg>
            <!-- Inline bean definition -->
            <bean class="Engine">
                <!-- Constructor args for inline Engine bean -->
                <constructor-arg value="V8 Petrol" />
                <constructor-arg value="400" />
            </bean>
        </constructor-arg>
    </bean>
</beans>
```

```

        <constructor-arg value="V12 Turbo" />
        <constructor-arg value="600" />
    </bean>
</constructor-arg>
</bean>

</beans>

```

Step 4 — MainApp.java (Spring container & application entry point):

```

// File: MainApp.java
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class MainApp {
    public static void main(String[] args) {

        System.out.println("Starting Spring Container...
");

        // Load Spring configuration from XML
        // Spring reads XML, creates beans, injects dependencies automatically
        ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        System.out.println("
--- Getting beans from container ---");

        // Get 'myCar' bean – Spring has already injected Engine into Car
        Car myCar = (Car) context.getBean("myCar");
        myCar.displayInfo();
        myCar.drive();

        System.out.println();

        // Get 'sportsCar' bean
        Car sportsCar = (Car) context.getBean("sportsCar");
        sportsCar.displayInfo();
        sportsCar.drive();

        // Close the container
        ((ClassPathXmlApplicationContext) context).close();
        System.out.println("
Spring Container closed.");
    }
}

```

Console Output:

```

Starting Spring Container...

Engine constructor called: V8 Petrol
Car constructor called: Toyota Fortuner
Engine constructor called: V12 Turbo
Car constructor called: BMW M Series

--- Getting beans from container ---

===== Car Information =====
Brand      : Toyota
Model      : Fortuner
Engine     : V8 Petrol
Power      : 400 HP
Driving Toyota Fortuner...
Engine started: V8 Petrol (400 HP)

===== Car Information =====

```

```
Brand      : BMW
Model      : M Series
Engine     : V12 Turbo
Power      : 600 HP
Driving BMW M Series...
Engine started: V12 Turbo (600 HP)
```

Spring Container closed.

Constructor Injection — Key Points:

Advantages:

- + Guarantees complete object initialization (all fields set at creation)
- + Dependencies are immutable (can use 'final' fields)
- + Makes testing easier (pass mock objects via constructor)
- + Clear documentation of required dependencies
- + Prevents circular dependency at startup (fails fast)

Disadvantages:

- Many dependencies = long constructor parameter list
- Cannot change dependency after object creation

constructor-arg attributes:

```
value="..." : literal value (String, int, boolean, etc.)
ref="..."   : reference to another Spring bean by id
type="..."  : specify type when ambiguous (e.g., type="int")
index="..."  : specify position (0-based) when ambiguous
```

Q6. Spring program demonstrating setter-based Dependency Injection.

Concept:

In **Setter Injection**, the Spring container calls setter methods (**setXxx()**) to inject dependencies AFTER creating the object with the no-arg constructor. Configured using **<property>** tags in XML. More flexible than constructor injection.

Step 1 — Address.java (Dependency):

```
// File: Address.java
public class Address {
    private String street;
    private String city;
    private String pincode;

    // No-arg constructor (required for setter injection)
    public Address() {}

    // Setters – Spring calls these to inject values
    public void setStreet(String street) { this.street = street; }
    public void setCity(String city)     { this.city   = city; }
    public void setPincode(String pincode) { this.pincode = pincode; }

    // Getters
    public String getStreet() { return street; }
    public String getCity()   { return city; }
    public String getPincode() { return pincode; }

    @Override
    public String toString() {
        return street + ", " + city + " - " + pincode;
    }
}
```

Step 2 — Student.java (Main bean with setter injection):

```
// File: Student.java (Spring bean)
import java.util.List;
```



```

<!-- Student Bean (main) – uses setter injection -->
<bean id="student1" class="Student">
    <!-- Primitive types -->
    <property name="id" value="1001" />
    <property name="name" value="Rahul Sharma" />
    <property name="gpa" value="8.75" />
    <property name="active" value="true" />

    <!-- Object reference injection -->
    <property name="address" ref="studentAddress" />

    <!-- List injection -->
    <property name="hobbies">
        <list>
            <value>Reading</value>
            <value>Coding</value>
            <value>Cricket</value>
        </list>
    </property>
</bean>

<!-- Second student bean -->
<bean id="student2" class="Student">
    <property name="id" value="1002" />
    <property name="name" value="Priya Patil" />
    <property name="gpa" value="9.10" />
    <property name="active" value="true" />
    <property name="address">
        <!-- Inline bean -->
        <bean class="Address">
            <property name="street" value="55 Station Road" />
            <property name="city" value="Pune" />
            <property name="pincode" value="411001" />
        </bean>
    </property>
    <property name="hobbies">
        <list>
            <value>Painting</value>
            <value>Dancing</value>
        </list>
    </property>
</bean>

</beans>

```

Step 4 — MainApp.java:

```

// File: MainApp.java
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class MainApp {
    public static void main(String[] args) {

        System.out.println("Loading Spring container...");

        ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        System.out.println("
--- Beans retrieved from Spring Container ---");

        // Get student1 bean
        Student s1 = (Student) context.getBean("student1");
        s1.displayInfo();
    }
}

```



```

        // Get student2 bean
        Student s2 = (Student) context.getBean("student2");
        s2.displayInfo();

        // Verify DI worked
        System.out.println("
Dependency Injection Verified:");
        System.out.println("s1 address city: " + s1.getAddress().getCity());
        System.out.println("s2 first hobby : " + s2.getHobbies().get(0));

        ((ClassPathXmlApplicationContext) context).close();
    }
}

```

Console Output:

```

Loading Spring container...

Student no-arg constructor called.
setId()   called with: 1001
setName() called with: Rahul Sharma
Student no-arg constructor called.
setId()   called with: 1002
setName() called with: Priya Patil

--- Beans retrieved from Spring Container ---

===== Student Information =====
ID       : 1001
Name     : Rahul Sharma
GPA      : 8.75
Active   : true
Address  : 101 MG Road, Solapur - 413001
Hobbies  : [Reading, Coding, Cricket]

===== Student Information =====
ID       : 1002
Name     : Priya Patil
GPA      : 9.1
Active   : true
Address  : 55 Station Road, Pune - 411001
Hobbies  : [Painting, Dancing]

Dependency Injection Verified:
s1 address city: Solapur
s2 first hobby : Painting

```

Constructor Injection vs Setter Injection — Final Comparison:

Feature	Constructor Injection	Setter Injection
Injection trigger	At object creation	After object creation
XML tag	<constructor-arg>	<property>
No-arg constructor	Not required	Required by Spring
Partial injection	Not possible	Possible (optional fields)
Immutability	Supports (final fields)	Does not support final
Circular dependency	Fails at startup	Resolved at runtime
Best for	Mandatory dependencies	Optional dependencies
Readability	Less readable (many params)	More readable (named setters)
Spring recommendation	Preferred (Spring team)	Use for optional deps

Spring XML Property Injection Types — Quick Reference:

```
<!-- String / primitive -->
<property name="name" value="Rahul" />
<property name="age" value="21" />
<property name="gpa" value="8.5" />
<property name="active" value="true" />

<!-- Object reference -->
<property name="address" ref="addressBeanId" />

<!-- List -->
<property name="hobbies">
  <list><value>Reading</value><value>Coding</value></list>
</property>

<!-- Set (no duplicates) -->
<property name="skills">
  <set><value>Java</value><value>Python</value></set>
</property>

<!-- Map (key-value pairs) -->
<property name="grades">
  <map>
    <entry key="Math" value="A" />
    <entry key="Science" value="B+" />
  </map>
</property>

<!-- Properties (String key-value) -->
<property name="config">
  <props>
    <prop key="db.url">jdbc:mysql://localhost/db</prop>
    <prop key="db.user">root</prop>
  </props>
</property>

<!-- Null value -->
<property name="email"><null /></property>
```